

Integrated Polar Science Outreach, Knowledge Repository & Media Dissemination Portal

Problem Statement ID: 26063 | **Organization:** Ministry of Earth Sciences (MoES) | **Department:** NCPOR | **Theme:** Smart Education

Purpose: Single-source implementation guidance covering UI pages, feature architecture, backend APIs, database choice, AI service design, search, knowledge graph, notifications, social outreach, education and administration.

1. Product Vision

- Build a clean scientific knowledge portal that archives polar reports, datasets, publications, research, photographs, videos and institutional activities while enabling AI-assisted discovery, summarization, trend analysis and outreach.
- Primary UX principle: Home is a discovery/intelligence surface; deeper functionality lives in the Sidebar and dedicated pages so the interface stays uncluttered.
- Core pillars: Polar Knowledge Repository, Semantic Search, AI Research Assistant, Knowledge Graph, Research Connections, AI Outreach Engine and Polar Science Intelligence.

2. Home Page

- Hero: prominent semantic search accepting natural-language questions such as research/topic/location queries.
- Quick Access: Expeditions, Reports, Scientific Datasets, Research, Publications, Photographs, Videos, Conferences, Seminars, Workshops and Awareness Events.
- Recent Uploads: latest reports, datasets, publications, photographs and videos with type, date, institution and actions.
- AI Data Trends: research activity, publication trends, expedition activity, dataset growth, topic frequency and emerging research areas.
- Knowledge Graph Preview: interactive researcher–institution–research–publication–dataset–expedition relationships.
- Featured Research: highlighted research/projects/publications selected by administrators or relevance.
- Upcoming Institutional Activities: conferences, seminars, workshops, awareness events and outreach programs.
- Notification icon at top-right: recent generated posts, new reports, dataset changes, new publications and activity updates.
- Authentication: Login/Sign Up when logged out; Profile icon when logged in.

3. Sidebar / Navigation

- Main: Home, Explore, Expedition Explorer, Knowledge Repository.
- Knowledge: Reports, Scientific Datasets, Research, Publications, Photographs, Videos.
- Outreach: Social Media Studio, Institutional Activities, Conferences, Seminars, Workshops, Awareness Events.
- Intelligence: AI Polar Intelligence, Knowledge Graph, Research Connections, Semantic Search.
- User: Saved Content, History, Profile and Settings.

4. Core Pages to Design

- Home: discovery, search, recent uploads, AI trends, graph preview and activities.
- Explore / Global Search: semantic search, filters, sorting, relevance scores and search history.
- Expedition Explorer: interactive Arctic/Antarctic map, expedition timeline and related content.
- Knowledge Repository: central content browser for all scientific and outreach assets.
- Reports: searchable report library with filters and AI summaries.
- Scientific Datasets: dataset catalog with metadata, versions, download and related research.
- Research: research projects, domains, researchers, institutions and related outputs.
- Publications: papers, authors, journals, DOI, year and related datasets/expeditions.
- Photographs: searchable polar image gallery with expedition/location/date metadata.
- Videos: expedition, educational, research, interview and institutional video library.

- Social Media Studio: generate, edit, approve, schedule and publish outreach content.
- Institutional Activities: conferences, seminars, workshops, awareness, training and exhibitions.
- Event Detail: date, venue, speakers, institution, description, registration and related research/media.
- AI Polar Intelligence: hub for AI Research Assistant, Summarizer, Metadata Generator, Trends and content generation.
- AI Research Assistant: repository-grounded conversational research interface.
- AI Document Analyzer: executive summary, key findings, methodology, entities, numbers and related research.
- Polar Science Intelligence: trend dashboards and AI-generated scientific insights.
- Knowledge Graph: full-screen interactive graph of scientific relationships.
- Research Connections: discover related researchers, institutions, publications, datasets and topics.
- Notifications: full notification center with read/unread state and categories.
- Profile: identity, institution, research interests, saved content and followed topics.
- Saved Content: saved reports, publications, datasets, expeditions and researchers.
- History: recently viewed content and searches.
- Polar Education: student-friendly polar science learning modules.
- Learning Topic Detail: simplified explanation, visuals, videos, related research and quizzes.
- Login / Signup / Password Reset: authentication flows.
- Settings: account, notifications, language, appearance, privacy and preferences.
- Admin Dashboard: repository, user, AI and engagement overview.
- Content Management: upload, edit, categorize, approve, publish and delete content.
- Dataset Management: metadata, versions, updates and change tracking.
- User Management: users, researchers, institutions, roles and permissions.
- AI Approval: review/edit/approve/reject AI-generated content before publication.
- Admin Analytics: search activity, content growth, dataset usage, AI usage and engagement.

5. Repository Content Model

- Expedition: location, dates, objectives, researchers, reports, datasets, media and publications.
- Report: title, authors, institution, expedition, region, year, document URL, summary, keywords and provenance.
- Dataset: title, description, variables, source, collection date, region, license, version and download location.
- Research: title, domain, researchers, institutions, publications, datasets and expeditions.
- Publication: title, authors, journal, DOI, year, research domain, datasets and expedition relationships.
- Media: type (photo/video), title, source, expedition, location, date, photographer/creator and storage URL.
- Institution: name, type, researchers, research, expeditions, publications and activities.
- Researcher: profile, institution, research areas, publications, datasets, expeditions and connections.
- Activity/Event: type, title, date, venue, institution, speakers, description and media.
- User: profile, role, institution, interests, saved items, history and notification preferences.

6. REST API Convention

- Use unversioned API URLs such as `/api/reports` rather than `/api/v1/reports` as requested.
- Use REST resources and query parameters for filtering instead of creating separate endpoints for every filter combination.
- Protect admin and mutation endpoints with authentication and role-based authorization.
- Return consistent JSON structures with success/error information, resource IDs and pagination metadata where appropriate.

7. Authentication APIs

- POST `/api/auth/signup` — create user account.
- POST `/api/auth/login` — authenticate user and issue session/access credentials.

- POST /api/auth/logout — terminate session.
- POST /api/auth/refresh — refresh authentication credentials where applicable.
- GET /api/auth/me — return current authenticated user.
- POST /api/auth/forgot-password — start password recovery.
- POST /api/auth/reset-password — complete password reset.

8. User APIs

- GET /api/users/me — retrieve current profile.
- PATCH /api/users/me — update current profile.
- GET /api/users/me/saved — list saved resources.
- POST /api/users/me/saved — save a resource.
- DELETE /api/users/me/saved/:contentId — remove saved resource.
- GET /api/users/me/history — retrieve browsing/search history.
- DELETE /api/users/me/history — clear history.
- GET /api/users/:id — retrieve public user/researcher profile where permitted.

9. Repository APIs

- Reports: GET/POST/PATCH/DELETE /api/reports and GET /api/reports/:id.
- Datasets: GET/POST/PATCH/DELETE /api/datasets, GET /api/datasets/:id, GET /api/datasets/:id/versions and /download.
- Publications: GET/POST/PATCH/DELETE /api/publications and GET /api/publications/:id.
- Research: GET/POST/PATCH/DELETE /api/research and GET /api/research/:id plus related resources.
- Expeditions: GET/POST/PATCH/DELETE /api/expeditions and related reports/datasets/media/research endpoints.
- Media: GET/POST/PATCH/DELETE /api/media with type=photo or type=video filters.
- Institutions: GET/POST/PATCH/DELETE /api/institutions and related research/researchers/activities.
- Researchers: GET /api/researchers, GET /api/researchers/:id and related publications/research/expeditions/connections.
- Activities: GET/POST/PATCH/DELETE /api/activities with conference/seminar/workshop/awareness/training filters.

10. Search, Graph & Research Connection APIs

- POST /api/search — semantic search using a natural-language query plus filters.
- GET /api/search/suggestions?q=... — search suggestions.
- GET /api/search/recent — recent searches for the authenticated user.
- GET /api/knowledge-graph — return graph nodes and edges.
- GET /api/knowledge-graph/node/:id — retrieve graph node details.
- GET /api/knowledge-graph/node/:id/relationships — retrieve connected nodes/edges.
- GET /api/connections/researchers/:id — discover researcher connections.
- GET /api/connections/research — discover research relationships.
- GET /api/connections/institutions/:id — discover institutional connections.
- POST /api/connections/discover — discover researchers/institutions/publications/datasets related to a topic.

11. AI APIs

- POST /api/ai/summarize — generate a repository-grounded report/document summary.
- POST /api/ai/ask — answer questions using repository content/RAG.
- POST /api/ai/generate-social-post — generate platform-specific outreach content.
- POST /api/ai/generate-metadata — extract/generate title, authors, keywords, domain, location and other metadata.
- GET /api/ai/insights — retrieve generated intelligence insights.
- GET /api/ai/trends — retrieve research/publication/expedition/dataset trends.

- POST /api/ai/recommend — generate personalized or context-based recommendations.
- Use asynchronous AI jobs for large documents: return jobId/status first, then retrieve results when complete.

12. Social Media APIs

- GET /api/social/posts — list generated/draft/published posts.
- GET /api/social/posts/:id — retrieve one post.
- POST /api/social/posts/generate — generate a post draft.
- PATCH /api/social/posts/:id — edit a draft.
- POST /api/social/posts/:id/approve — approve content after human review.
- POST /api/social/posts/:id/schedule — schedule approved content.
- POST /api/social/posts/:id/publish — publish through configured platform integration.
- DELETE /api/social/posts/:id — remove a draft/post record.
- Workflow: AI Generate → Draft → Human Review → Approve → Schedule/Publish.

13. Notifications APIs

- GET /api/notifications — list notifications.
- GET /api/notifications/unread — list unread notifications.
- PATCH /api/notifications/:id/read — mark one notification as read.
- POST /api/notifications/read-all — mark all notifications as read.
- DELETE /api/notifications/:id — delete notification.
- Notification types include new content, dataset changes, AI output, events and followed-topic/research updates.

14. Education APIs

- GET /api/education/topics — list learning topics.
- GET /api/education/topics/:id — retrieve topic content.
- GET /api/education/topics/:id/related — retrieve related scientific resources.
- GET /api/education/quizzes/:id — retrieve quiz.
- POST /api/education/quizzes/:id/submit — submit quiz answers.

15. Admin APIs

- GET /api/admin/dashboard — admin summary.
- GET /api/admin/analytics — admin analytics.
- GET /api/admin/content/pending — pending moderation queue.
- POST /api/admin/content/:id/approve — approve content.
- POST /api/admin/content/:id/reject — reject content.
- GET /api/admin/users — list/manage users.
- PATCH /api/admin/users/:id/role — update user role.
- GET /api/admin/ai/pending — AI content requiring review.
- POST /api/admin/ai/:id/approve — approve AI content.
- POST /api/admin/ai/:id/reject — reject AI content.

16. Database Decision

- Primary database: PostgreSQL because the portal has many structured relationships between researchers, institutions, expeditions, reports, datasets, publications, events and users.
- Use PostgreSQL + pgvector for semantic embeddings so relational data and vector search can coexist in the initial architecture.
- Do not introduce MongoDB unless a specific unstructured-data requirement emerges; it is not necessary for the first version.
- Do not introduce Neo4j initially; build the Knowledge Graph from PostgreSQL relationships and return nodes/edges to the frontend.

- Use object storage such as S3, R2, Supabase Storage, Azure Blob or MinIO for PDFs, high-resolution photographs, videos and large datasets instead of storing binary files directly in PostgreSQL.
- Dataset versioning is allowed and useful even though API URLs remain unversioned; API versioning and dataset versioning are separate concepts.

17. AI Service Decision

- Create a logically separate Python FastAPI AI service, but keep deployment simple rather than building a complex microservice ecosystem on day one.
- Node.js/Express is the application brain for authentication, CRUD, permissions, repository APIs, notifications and social workflows.
- Python/FastAPI is the intelligence/scientific-processing brain for LLM calls, embeddings, RAG, document processing, metadata extraction and trend analysis.
- Frontend should call Node only; Node should call the AI service so AI infrastructure and provider credentials are not exposed to the browser.
- Recommended flow: React → Node API → Python AI Service → LLM/Embedding/Vector Layer.

18. AI Service Structure

- ai-service/app.py — FastAPI application entry point.
- routes/summarize.py — document summarization endpoints.
- routes/embeddings.py — embedding generation and indexing.
- routes/trends.py — trend/insight generation.
- routes/knowledge_graph.py — entity/relationship extraction.
- routes/social.py — outreach content generation.
- services/llm.py — LLM provider abstraction.
- services/embeddings.py — embedding model abstraction.
- services/rag.py — retrieval-augmented generation pipeline.
- services/document_processor.py — PDF/OCR/text extraction and chunking.

19. Semantic Search / RAG Flow

- User enters natural-language query in the frontend.
- Node receives query and filters and forwards the AI/search request.
- Python creates a query embedding and performs vector similarity search using pgvector.
- Relevant reports, publications, datasets and research records are retrieved.
- RAG assembles grounded context and sends it to the LLM.
- AI returns an answer plus source/resource references so users can inspect the underlying scientific content.

20. Document AI Pipeline

- Upload document → object storage → metadata record → processing job → PDF/text/OCR extraction → chunking → embeddings → pgvector → summary/metadata/entities → notification.
- Large document processing should be asynchronous and represented by a jobId/status instead of blocking the browser for a long request.
- Generated summaries and metadata should be stored with provenance including model, timestamp and source document ID.

21. Knowledge Graph Strategy

- Represent entities such as researchers, institutions, expeditions, reports, datasets, publications, topics and locations.
- Represent relationships such as authored, works-at, participated-in, produced, related-to, collected-at and studies.
- Generate graph JSON as nodes and edges for frontend visualization using a graph library such as React Flow or Cytoscape.js.
- Move to Neo4j only if relationship traversal/graph scale later justifies a dedicated graph database.

22. Backend Folder Structure

- backend/src/config — environment/database configuration.
- backend/src/middleware — authentication, authorization, validation and error handling.
- backend/src/utils — shared utilities.
- backend/src/modules — auth, users, reports, datasets, publications, research, expeditions, media, institutions, researchers, activities, search, graph, connections, notifications, social, education and admin.
- backend/src/ai — Node-side AI orchestration/client and AI routes.
- backend/src/app.js — Express app configuration.
- backend/src/server.js — server startup.

23. Recommended Final Technology Stack

- Frontend: React or Next.js.
- Main backend: Node.js + Express.
- Primary database: PostgreSQL.
- Semantic vector layer: pgvector.
- AI service: Python + FastAPI.
- LLM: selected external LLM provider behind a provider abstraction.
- File storage: S3/R2/Supabase Storage/Azure Blob/MinIO.
- Background jobs: Redis + BullMQ later when asynchronous processing grows.
- Knowledge Graph: PostgreSQL relationships initially.
- Graph visualization: React Flow or Cytoscape.js.
- Authentication: secure HTTP-only session/cookie approach or short-lived access tokens with refresh strategy.

24. Recommended MVP Scope

- Phase 1: authentication, Home, repository, reports, datasets, publications, expeditions and basic media.
- Phase 2: semantic search, pgvector, AI summarization and AI research assistant.
- Phase 3: knowledge graph, research connections and AI trends.
- Phase 4: social media studio, notifications, institutional activities and education mode.
- Phase 5: admin moderation, analytics, data provenance, version history and production hardening.
- Keep the first deployment simple: PostgreSQL + pgvector + object storage + Node API + Python AI service.

25. Key Product Principles

- Keep Home clean and discovery-focused rather than showing every feature at once.
- Use AI as an assistant and accelerator, not as the sole authority; provide source references and human approval for public-facing generated content.
- Scientific content should expose provenance, source, authorship, institution, update time and dataset version where available.
- Use consistent REST resources and query parameters; do not create endpoint variants for every filter.
- Design API contracts before frontend integration so every page has a predictable data source.
- Prefer modular architecture now so components can later be separated into independent services without rewriting the entire system.